

Report (7368 words, excluding tables, figures, references, and appendices)

1. Introduction (482/1000 words)

When computing researchers do a literature review, they often need to read many papers and find each paper's method, task, dataset, and evaluation metric. Reading many papers this way is slow and manual. I treat these four items as a methodology profile that a reader can extract automatically, to support the first pass of a literature review, not to replace reading the paper.

Consider "Attention Is All You Need" [D6]:

Methodology role	Component
Technical method	Transformer
Task	machine translation
Dataset	WMT 2014 English-German
Evaluation metric	BLEU

Table 1: Role-based methodology profile for "Attention Is All You Need" [D6].

A reader needing this summary currently has to read the paper and construct it themselves. A topic label answers "what is this paper about?"; a role-based profile answers "how was this research conducted?" Two papers on the same topic can use different methods, train on different datasets, and report different metrics, so a topic label alone does not answer that second question.

I use Template 12.1 from the Natural Language Processing (NLP) module: identifying research methodologies used in computing research papers. The code repository is publicly available at <https://github.com/sanemat/uol-fp>.

This motivation has not changed since the preliminary report. What changed is the extraction approach. proto2, my first working prototype, classified every sentence in a paper into one of the four roles using zero-shot natural language inference (NLI), producing a list of candidate sentences per role rather than one answer. proto3, the current prototype, reframes the task as document-level extraction: given a paper, a long-context large language model (LLM) returns one answer per role, each backed by a section heading and a verbatim quote as evidence. I also implemented a decomposed variant of proto3 (four role-specific calls instead of one joint call) and a series of pilots aimed at Task, the weakest role.

The primary users are computing students doing literature reviews. Secondary users are early-stage researchers or supervisors who want a quick overview of a paper. The output is designed to be inspectable: a user can check the quoted evidence against the source paper.

Report structure

Chapter 2 reviews related work on how research methodology is defined and structured, how methodology-related information can be extracted from scientific papers, and how zero-shot classification assigns labels without labeled training data, then extends this with work on LLM-based structured extraction, and with proto2's own findings as motivation for document-level extraction.

Chapter 3 describes the system design: the four-role schema and its justification, the joint and decomposed extraction variants, the pipeline architecture, the model choice, and the evaluation plan.

Chapter 4 covers the whole implementation: proto2, proto3's document-level extraction (pipeline stages, schema-guided prompt, Gemini call), the decomposed variant, and the Task pilots.

Chapter 5 evaluates the whole project: gold-label matching scored as a classification problem with confidence intervals, a logged five-run variance study, a manual review of the evidence, the decomposed-extraction and Task experiments, and a critical evaluation.

Chapter 6 concludes with a short summary of the project and further work, including one broader theme about structured-output guarantees versus semantic correctness in LLM-based extraction.

2. Literature Review (1490/2500 words)

Chapter 1 showed a four-role profile for "Attention Is All You Need" [D6]. Figure 1 shows a fuller view of the same paper, including the design strategy and data generation method defined by Oates [14].

Methodology:

Design or strategy: design and creation + experiment

Data generation method: documents

Technical method: Transformer

Task: machine translation

Dataset: WMT machine translation datasets

EvaluationMetric: BLEU score

Figure 1: Research Methodology from "Attention Is All You Need" [D6].

This review covers four areas: how methodology is defined, how information is extracted from papers, how classification can work without labeled training data, and how large language models (LLMs) extract structured information directly from scientific text. It ends with a synthesis positioning this project.

1. Defining Research Methodology

Research methodology in computing papers can be described using a structured vocabulary, but defining it is not the same as extracting it.

Oates [14] provides six research strategies (experiment, design and creation, survey, case study, action research, and ethnography) and four data generation methods (interviews, observations, questionnaires, and documents). His book defines the vocabulary that researchers use to describe their methodology in papers, so my

project needs these concept names to identify what to extract. However, the six strategies were designed for human researchers to self-classify their own work — papers rarely contain the explicit phrase "this is an experiment". The vocabulary can be used to name what to look for, but it may not transfer directly to automatic extraction from text.

Pilkington & Pretorius [15] go further: they formalize the structure using UML (Unified Modeling Language) and ontology engineering, with the goal of "providing clear and unambiguous semantics" [15]. Key concepts are ResearchScheme, PhilosophicalWorldview, ResearchDesign, and ResearchMethod: a ResearchScheme is underpinned by one PhilosophicalWorldview and has one or more ResearchDesigns, and each ResearchDesign has one or more ResearchMethods.

A philosophical worldview is one of the key components in Pilkington & Pretorius [15], but it tends not to appear as an explicit phrase in completed research papers, so I exclude it from extraction. Research design (e.g. experiment vs. survey) [10] is similarly out of scope: it can be subjective, two readers can assign different labels to the same paper, and it is not the focus of this project's four roles.

Oates [14] gives concept names. Pilkington & Pretorius [15] give formal relationships between those concepts. My project uses vocabulary from Oates and formal structure from Pilkington & Pretorius. Both works are designed for human use; neither provides a system to extract methodology components automatically from text. The four roles used in this project (TechnicalMethod, Task, Dataset, and EvaluationMetric) draw more directly on Jain et al. [8], reviewed next.

2. Extracting Methodology from Papers

Systems that extract methodology-like entities from papers exist [4, 5, 8], but the closest approaches are supervised and need labeled training data that I do not have.

Jain et al. [8] (SciREX) extract four entity types — Dataset, Metric, Task, and Method — that closely match the four roles in this project. They operate at the document level, arguing that "a significant amount of information can only be gleaned from analyzing the full document" [8] — relations may span sections, not just sentences. However, Jain et al. annotated 438 papers with a main annotator, and four PhD-level experts checked agreement on five documents (Cohen's κ 95% for mention classification). The annotation builds on the Papers with Code corpus of 1,170 ML-conference articles, which covers only ML benchmarks. My project targets general computing papers (systems, algorithms, human-computer interaction (HCI), and ML research) where no comparable annotated dataset is available, so neither the corpus scope nor the annotation effort behind SciREX transfers directly.

Ma et al. [13] propose a metric-driven mechanism schema in which a mechanism is an (operation, effect, direction) triple linked to a task, and extract it from ACL abstracts using a query-guided sequence-to-sequence model, but their work is limited to the NLP domain and does not extend to general computing research.

Ghosh et al. [4, 5] use supervised transformer-based sequence labeling to extract methodology component names from AI research papers. They argue that methodology names are difficult to extract because they are large, fast-evolving, domain-specific, and context-dependent. Unlike Jain et al. [8], who extract four entity types at the document level, Ghosh et al. focus narrowly on TechnicalMethod. Their approach covers only TechnicalMethod, leaving Task, Dataset, and EvaluationMetric unaddressed, and their training data does not cover general computing research such as systems, algorithms, or HCI.

Färber et al. [3] extract methods and datasets from scientific publications using domain-specific named entity recognition (NER) followed by usage classification, which distinguishes whether each mention is used by the authors or only mentioned. This "used vs. non-used" distinction is directly relevant to a noise problem this project also has: a sentence describing a prior method (for example, ELMo,

cited in BERT's Introduction) can be misread as the paper's own method. However, Färber et al. cover only Method and Dataset, not Task or EvaluationMetric, and their NER model requires labeled entity mentions that are not available for this project.

These supervised methods all depend on domain-specific annotated corpora that this project does not have. A zero-shot approach removes this requirement, at the cost of domain adaptation, reviewed next.

3. Zero-shot Classification

Yin et al. [18] define zero-shot text classification as assigning a label to text without any task-specific training examples. They show that natural language inference (NLI) can classify text into many possible labels by turning the label into a hypothesis (for example, "this text is about sports") and asking a model whether the text entails it. I applied this approach directly in proto2, my first prototype: four methodology-role hypotheses, one per role, classified independently against each sentence in a paper, using a DeBERTa-v3-based zero-shot classification model [6].

aspect t	labels	interpretation	example hypothesis (word)
topic	sports etc.	this text is about ?	"?" = sports
emotion	anger etc.	this text expresses ?	"?" = anger
situation	shelter etc.	The people there need ?	"?" = shelter

Table 2 (adapted from Yin et al. [18]): example hypotheses for three general-domain task types.

A domain mismatch risk exists: Yin et al. test on Yahoo News articles, emotion tweets, and crisis situation reports, and their NLI model is trained on MNLI (Multi-Genre Natural Language Inference; covers

news, fiction, and telephone speech) — none of these are scientific papers, which tend to use dense technical vocabulary, passive constructions, and section-based structure. proto2 tested this risk directly by comparing hypothesis wordings on scientific text: a verbose TechnicalMethod hypothesis sent 244 of 258 BERT sentences to EvaluationMetric, a single poorly chosen hypothesis collapsing the label distribution, while short labels gave the best probe score and the most balanced distribution across four roles. Even with short labels, classification errors remained: on the Transformer paper, "Our model achieves 28.4 BLEU on the WMT 2014 English-to-German translation task" was classified as Task rather than EvaluationMetric. Hypothesis design is therefore a real cost in zero-shot systems, and it did not fix the design-level problem in proto2's own results, covered next.

4. proto2's Own Findings as a Negative Result

Sentence-level NLI classification produced too many candidate sentences to be usable — 151 TechnicalMethod sentences for the MapReduce paper alone — and its recall-only substring evaluation (18/24 across six papers) only checked whether a gold term appeared somewhere in the output, not whether the output itself was correct. With 100+ accepted sentences in some roles, a substring match is nearly certain to succeed somewhere in the list, which inflates the apparent recall without saying anything about precision. Excluding Related Work by heading, which describes other papers rather than the target paper's own methodology, cut BERT's TechnicalMethod count from 67 to 62 sentences; excluding the whole Introduction cut it further to 54, but also removed sentences that correctly described BERT's own method, so full exclusion traded recall for precision rather than solving the underlying problem. proto2 also had no mechanism to separate a paper's own method from one it cites: a sentence describing ELMo in BERT's Introduction scored 0.87 as TechnicalMethod, even though ELMo is prior work, not BERT's own method. This sharpens Färber et al.'s [3] "used vs. non-used" gap (Section 2 above) with a concrete instance from my own data, and

motivates proto3's document-level extraction with an explicit authorship rule (Chapter 3).

5. Document-Level and LLM-Based Structured Extraction

Jain et al. [8] argue that "a significant amount of information can only be gleaned from analyzing the full document" — a document-level information extraction (IE) claim. My own data supports this directly: Dataset and EvaluationMetric typically appear only in a paper's Experiment section, not the Abstract, so an extraction method effectively limited to a small set of sentences, as proto2's Introduction-heavy output tended to be, would miss them.

Structured extraction with LLMs is an established research approach. Dagdelen et al. [2] extract structured information from scientific text with LLMs. Polak and Morgan [16] extract materials data from research papers using conversational LLMs and prompt engineering. Ateia et al. [1] apply LLMs to extract information from scientific papers, the closest match in domain to this project. All three pull structured fields out of scientific documents, similar in spirit to how proto3 pulls a four-role methodology profile from a computing paper. I apply this general approach to this project's specific four-role schema, combined with the authorship rule (Section 4 above) and a document-level context window (Chapter 3).

6. Synthesis

I could not find prior work combining the structured methodology vocabulary from Oates [14] and Pilkington & Pretorius [15], the four-role schema from Jain et al. [8], zero-shot or LLM-based extraction without an annotated corpus, and general computing papers rather than only ML benchmarks, which motivated testing the combination directly across two prototype iterations.

Source	Contribution	Strength	Limitation	Relevance to this project
Oates [14]	Methodology	Clear concept	Designed for	

Source	Contribution	Strength	Limitation	Relevance to this project
	vocabulary	names for methodology components	human use; not extraction-oriented	Motivates structured extraction
Pilkington & Pretorius [15]	Formal methodology ontology	Formal relationships between concepts	No extraction system or corpus	Supports treating methodology as a structured domain
Jain et al. [8]	Document-level extraction of Dataset, Metric, Task, and Method	All four roles; working system; document-level argument	438 annotated ML papers; expert annotators	Confirms four roles; motivates document-level extraction and both proto2 and proto3's approaches
Ghosh et al. [4, 5]	TechnicalMethod extraction from AI papers	Methodology-specific sequence labeling	One role only; AI papers; supervised	Shows difficulty of extracting method names
Färber et al. [3]	Used vs. non-used methods and datasets	Handles authorship attribution	Method and Dataset only; labeled mentions required	Motivates proto3's authors'-own-work rule
Yin et al. [18]	Zero-shot NLI text	No task-specific	Tested on general-	Basis for proto2;

Source	Contribution	Strength	Limitation	Relevance to this project
	classification	training data needed	domain text only; sentence-level	superseded by document-level extraction in proto3
Dagdelen et al. [2]; Polak and Morgan [16]; Ateia et al. [1]	LLM-based structured extraction from scientific text	Document-level, schema-guided, no annotated corpus needed	Applied to other domains or to unstructured key concepts	Basis for proto3's schema-guided LLM extraction

Table 3: Key sources for this project.

3. Design (1311/2000 words)

The system extracts research methodology from computing papers. The end-to-end input is a PDF, which a local GROBID [11] server converts to TEI XML before the notebook pipeline starts. The output is a role-based profile (Table 1, Chapter 1).

1. Domain and Users

The domain and users are unchanged from the preliminary report. The domain is computing research papers, mainly systems, machine learning (ML), algorithms, and human-computer interaction (HCI). The primary users are computing students doing literature reviews; secondary users are early-stage researchers or supervisors who want a quick overview of a paper. proto3 changed the output quality — one checkable answer per role instead of a list of candidate sentences — not the target domain or audience.

Table 4 connects each user need to a system requirement and the metric that evaluates it, so that each requirement follows from a user need and can be measured.

User need	System requirement	Evaluation
Quickly understand a paper	Show the four roles clearly	Output size (one answer per role, Chapter 4)
Check the original paper	Show evidence sentences	Evidence correctness (manual review, Chapter 5 §4)
Avoid missing important information	High recall	Recall (Chapter 5 §2-3)
Avoid too much irrelevant information	Reduce noisy output	Precision (Chapter 5 §2-3)

Table 4: User needs mapped to system requirements and evaluation.

2. Design Justification

The core feature is Stage 2: one structured, evidence-backed answer per role, not a list of 14-160 candidate sentences (Chapter 2, Section 4). proto2 classified each sentence independently with a fixed 0.5 threshold, which assumed that a sentence carries at most one role and that one threshold suits all roles. proto3 gives the model the whole document and asks for one answer per role, with a section heading and a verbatim quote as evidence, so a reader can check the answer without reading the paper. The authors'-own-work rule in the prompt targets proto2's authorship-attribution failure (the ELMo/BERT case, Chapter 2, Section 4). The four-role JSON shape is enforced by `response_json_schema`, generated from Pydantic models rather than described in the prompt text, which removes an entire class of parsing and output-shape bugs (Chapter 4). A role can be null, so the model can report a role as absent instead of inventing one.

Extraction can be joint or decomposed. Variant A, the main pipeline, makes one call that returns all four roles; Jain et al.'s [8] document-level argument (Chapter 2) favours this design, since one context lets

the model link a method, dataset, and metric that appear together (e.g. "Transformer"/"WMT"/"BLEU"). Variant B makes four independent role-specific calls (Stage 2d). Khot et al. [9] show that decomposing a complex task into independently optimizable subtasks can beat a single joint few-shot prompt on several reasoning tasks, and the four roles ask for different judgments: primary method versus component, the problem actually solved, data actually used, and the metric actually reported. Both variants use the same RoleExtraction schema and the same scoring code, so they differ only in the number of calls and in one role-specific rule line. My hypothesis is that per-role accuracy under Variant B exceeds Variant A, which Chapter 5 §5 tests. Variant C, a fifth call checking the four outputs for mutual consistency, is designed but not implemented (Chapter 6).

The second design question is whether every role should stay single-valued. AlexNet and ResNet both report top-1 and top-5 error rates, which the baseline squashes into one string, and BERT's gold EvaluationMetric label lists both "accuracy" and "F1" (Appendix A, Table A1), since the paper reports both. An informal NotebookLM cross-check, run independently on each paper without being told the schema was single-valued, produced similar multi-valued outputs for BERT, ResNet, and Transformer. BERT's single-valued Dataset answer is "SQuAD v1.1," while NotebookLM listed BooksCorpus, Wikipedia, GLUE, and SQuAD v1.1/v2.0 from the same source text. Forcing Dataset and EvaluationMetric into one string loses information.

Task shows the same pattern in one case: Variant B answered "sequence transduction" for Transformer where Variant A answered "machine translation", two defensible answers at different granularity (Chapter 5 §5). Stage 2e therefore implements a multi-valued pilot for Task only (Chapter 4), followed by verification and reasoning-first variants. The main design keeps one answer per role. A multi-valued schema for Dataset and EvaluationMetric would need per-item evidence rather than one shared quote per list, and a ranked "primary first" order rather than a numeric confidence field, since this project's own measured non-determinism (Chapter 5) argues against a second, uncalibrated confidence axis. That schema remains a design proposal (Chapter 6).

3. Model Choice

A paper's cleaned full text is typically 4,000-20,000 tokens, which fits within the context window of several modern long-context LLMs without chunking:

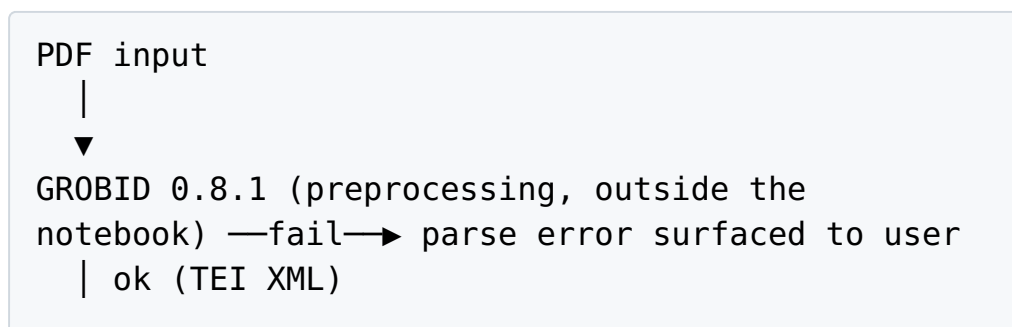
Model	Context	Cost
Gemini Flash	1M tokens	cheap API

Table 5: Long-context model.

I selected Gemini (gemini-3.5-flash, via the google-genai software development kit), one of the current general-purpose top models with a long context window. The aim of this project is to establish whether schema-guided LLM extraction is effective for this task, not to identify the best-performing model, so testing one such model is sufficient. Whether a more suitable model exists is left to further work. Its 1M-token context window exceeds the length of every paper in the corpus, so no chunking or truncation logic was needed. The API key comes from Colab's built-in secret manager (google.colab.userdata), which requires no account beyond the Google account already used for Colab. A comparison of model quality, cost, and reproducibility across providers is left to further work (Chapter 6).

4. Overall Pipeline

The pipeline has one preprocessing step outside the notebook and five stages inside it. Figure 2 shows data flow, component interaction, failure handling, and where results reach the user, including each stage's failure path and the final user-facing step:



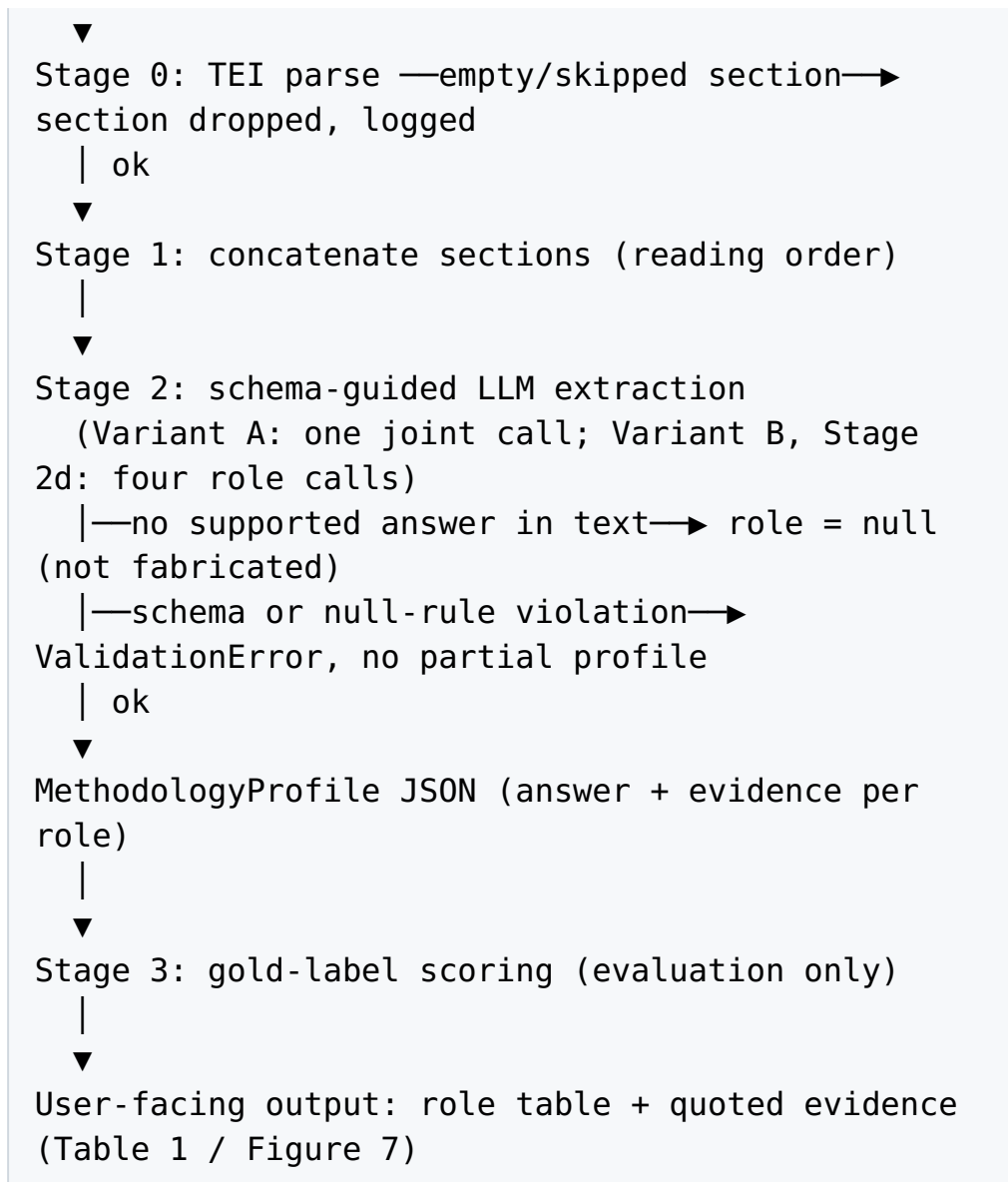


Figure 2: proto3 extraction pipeline with failure paths and the user-facing output step.

Compared with proto2's pipeline, there are two differences: there is no sentence splitting, and there is no per-sentence acceptance threshold. The LLM sees the (mostly) whole document and returns one decision per role directly, instead of a list of candidate sentences each scored independently. Figure 3 shows the two pipelines side by side.

proto2 (sentence level)
PDF → GROBID → TEI XML → section filter →
sentence split + clean

```
→ zero-shot NLI per sentence → threshold 0.5
→ 14-160 candidate sentences per role

proto3 (document level)
PDF → GROBID → TEI XML → concatenate sections
(reading order)
→ one LLM call + response_json_schema → one
answer + evidence per role
```

Figure 3: proto2 and proto3 pipelines compared. proto3 removes sentence splitting and the per-sentence acceptance threshold.

These two differences remove two weaknesses of proto2. First, proto2's NLI acceptance threshold of 0.5 needed a justification. proto3 has no such value, because extraction is no longer a per-sentence accept/reject decision. Second, proto2 assumed that a sentence describes one role at a time, and single-label sentence classification could only ever produce single-label results, whatever the underlying text actually contained. That circularity does not apply to proto3, since extraction is document-level; whether a *role* (not a sentence) should allow more than one answer is addressed in Section 2 above.

5. Evaluation Plan

proto2's plan treated a substring gold-label match with a 10-out-of-12 success threshold as sufficient, but a present-but-wrong answer cost nothing there. For proto3 I score gold-label match as a classification problem (true positive, false positive, false negative), report Precision, Recall, and F1 per role, and add Wilson 95% confidence intervals on Precision and Recall only — F1 is a harmonic mean, not a proportion, so a Wilson interval on it directly is not statistically meaningful, and a point-estimate F1 is appropriate at this sample size. I report both micro and macro averages, headlining macro, because the four roles are fixed, equally mandatory schema fields, not a frequency distribution.

I kept the sample at six papers: tightening the confidence intervals meaningfully would need roughly 30-40 gold-labeled papers per role, not the 6-10 reachable in the available time with no second annotator. This limits generalisability and remains a limitation of the evaluation. The plan also includes a logged variance study (repeat the pipeline several times rather than trust one run) and a single consolidated manual review pass covering plausibility, evidence support, authorship, and whether the quote appears in the source text.

I did not pool the five runs' true/false positive/negative counts into a single Wilson interval ($n=30$ trials per role): the 30 trials are five repeats of the same six papers, not 30 independent observations, so treating them as independent Bernoulli trials would overstate precision. The two measures stay separate: the $n=6$ baseline Wilson interval for paper-level uncertainty, and the five-run F1 mean, minimum, maximum, and range for run-to-run non-determinism. Each answers a different question, and neither substitutes for the other.

For Task, the evaluation adds a comparison of Variant A and Variant B (one run each, scored with the unchanged `score_role`), and a sequence of pilots whose F1 is compared against the same gold labels (Chapter 5 §5).

6. Iterations and Results

The project ran in iterations. Table 6 lists what each iteration produced and its status.

Period	Main task	Output	Status
Before 29 June	Literature review, design, proto2 (sentence-level NLI)	Preliminary Report	Done
July	proto3 Stages 0-2 (document-level extraction); gold-label-	Baseline P/R/F1 table, 5-run F1	Done

Period	Main task	Output	Status
	match evaluation with Wilson confidence intervals on Precision/Recall	variance table (mean/min/max/range)	
July-August	Consolidated manual review pass; proto2 → proto3 fixed/not-fixed synthesis; figures for Implementation/Evaluation chapters	Draft Report	Done
Late August	Variant B (decomposed extraction), scored against Variant A	Table 13	Done
Late August	Task pilots (Stages 2e-2h) and LLM-judge rescoring (Stage 4)	Table 14	Done
Late August	Variant C; Related Work ablation	—	Not run
September	Freeze experiments; final report; video	Final submission	In progress

Table 6: Iteration summary.

4. Implementation (1171/2500 words)

proto2 was my sentence-level zero-shot natural language inference (NLI) classifier: it classified every sentence in a paper into one of the four roles, producing a list of 14-160 candidate sentences per role rather than one answer. proto3 reframes the task as document-level extraction: given the TEI XML of a computing paper, it extracts one answer per role — TechnicalMethod, Task, Dataset, EvaluationMetric

— each with a section heading and a verbatim quote as evidence, using a schema-guided prompt to a long-context LLM (gemini-3.5-flash). On "Attention Is All You Need" [D6], for example:
TechnicalMethod = "Transformer", Task = "machine translation",
Dataset = "WMT 2014 English-German", EvaluationMetric = "BLEU",
each backed by its own quote and section.

Around this core I implemented the decomposed variant (Variant B, four calls per paper), four pilots aimed at the weak Task role (Stages 2e-2h), an LLM-judge rescoring step (Stage 4), and an evaluation harness (scoring.py, aggregate_runs.py, aggregate_variant_b.py) with pytest tests. The code is at <https://github.com/sanemat/uol-fp>.

1. Features Implemented

The prototype takes GROBID TEI XML and produces one JSON object containing an answer and evidence for each of the four roles (full example below). Parsing the XML and extracting section text reuses proto2's approach; the core new feature is extracting one structured answer and its evidence per role, using a schema-guided prompt to a long-context LLM rather than classifying each sentence independently. Every run can be scored against gold labels (Stage 3), and every answer can be checked against its quoted evidence.

2. Algorithms and Techniques

The end-to-end input is a PDF. A local GROBID 0.8.1 server converts it to TEI XML before the notebook runs (Appendix B), because GROBID keeps the section structure that Stages 0-1 need. This step is outside the notebook pipeline (proto3/3pipeline.ipynb), which takes the TEI XML as its input.

Stage 0, parse the TEI XML. `xml.etree.ElementTree` reads the file with the TEI namespace. The Abstract is taken from `tei:abstract`. Body sections are the `tei:div` elements under `tei:body`, with the heading from `tei:head` and the text from the `tei:p` children. Sections headed "references", "acknowledgements", or "acknowledgments" (case-insensitive) are skipped, as are sections with an empty body. The Abstract becomes the first section.

Stage 1, concatenate. Each section becomes `## {heading}` followed by its text, joined in reading order into one document. There is no sentence splitting, no chunking, and no per-sentence threshold, a direct response to proto2's output-volume problem (Chapter 2). Dataset and EvaluationMetric often occur only in the Experiment section, so I keep the full document rather than an excerpt.

Stage 2, schema-guided extraction. The prompt states the four role definitions and three rules. The four-role JSON shape is not described in the prompt. It is passed as `response_json_schema`, generated from the MethodologyProfile Pydantic model, with `temperature=0` and `seed=0`, and the reply is parsed with `MethodologyProfile.model_validate_json`. `extra="forbid"` rejects unknown fields, and a `model_validator` rejects a role where only one of answer and evidence is null.

Stage 2d, decomposed extraction (Variant B). Four independent calls, one per role, each with a role-specific prompt and the smaller RoleExtraction schema. Each prompt names the failure that role is prone to: TechnicalMethod must be the primary method, not a component or prior work; Task must be the problem the paper actually solves; Dataset must be used for training or evaluation, not only mentioned; EvaluationMetric must report the paper's own results. The four results are assembled into the same `{role: answer}` shape that Stage 3 reads.

Stages 2e-2h, Task pilots. Stage 2e uses MultiValuedRoleExtraction, a list of answers at different granularities with the primary answer first. Stage 2f asks a second call to select one candidate from that list. Stages 2g and 2h use ReasoningFirstRoleExtraction, in which a reasoning field precedes answer, following the "Thought of Structure" idea of Lu et al. [12] (reasoning before structured output) adapted to prompt time; 2g selects from Stage 2e's candidates and 2h extracts directly.

Stage 3, evaluation. `score_role` returns (tp, fp, fn, tn) per paper and role. A match is a normalised (lowercase, whitespace-collapsed) substring test in either direction. A both-present mismatch

counts as one false positive and one false negative, and a null answer against a non-null gold label counts as a false negative only. `score_role_multi` accepts a match at any list position, and `precision_recall_f1` computes the metrics from the sums.

Stage 4, LLM judge. `score_role_judged` takes the match test as an injected function, so a Gemini semantic-equivalence check (returning a `SameTaskVerdict`, one boolean field) can replace the substring test while the tests use a stub and need no API call.

3. Code Explanation

Four details are the most technically interesting. First, the prompt states only what the JSON schema itself cannot express, and everything about the output shape lives on the Pydantic models (Figure 4):

```
class Evidence(BaseModel):
    section: str = Field(description="Exact
section heading containing the quote.")
    quote: str = Field(
description="One sentence quoted verbatim from
the paper, supporting answer."
    )

class RoleExtraction(BaseModel):
    answer: str | None
    evidence: Evidence | None

    @model_validator(mode="after")
    def answer_and_evidence_must_match(self) -> S
elf:
        if (self.answer is None) != (self.evidenc
e is None):
            raise ValueError(
                "answer and evidence must either
both be null or both be present"
```

```
)  
return self
```

Figure 4: Evidence/RoleExtraction Pydantic models with the null-correlation validator (``proto3/src/uol_fp/models.py``).

The validator enforces "no answer without evidence" in code. A JSON schema alone cannot express this rule, so a prompt instruction would be the only alternative, and it would be unenforced. The models live in `models.py`, and `sync_generated.py` (make sync-generated) generates the notebook block from that file, so the notebook and the tested module cannot drift apart.

Rules:

- Use the authors' own method, not methods cited from prior work.
- Return null when a role is not present in the paper.
- Evidence quotes must be copied verbatim from the paper, not paraphrased.

Figure 5: Prompt rules excerpt from ``proto3/3pipeline.ipynb``, "Stage 2b — Prompt Template".

Second, the Gemini call uses `response_json_schema` together with `temperature=0` and `seed=0`, and parses the reply directly with `MethodologyProfile.model_validate_json(...)`, with no manual JSON-extraction step:

```
response = client.models.generate_content(  
    model=MODEL_NAME,  
    contents=prompt,  
    config=types.GenerateContentConfig(  
        temperature=0,  
        seed=0,  
        response_mime_type="application/json",  
        response_json_schema=MethodologyProfile.m  
odel_json_schema(),  
    ),
```

```
)

profile =
MethodologyProfile.model_validate_json(response.text)
```

Figure 6: Gemini call and response parsing
(`proto3/3pipeline.ipynb`, "Stage 2c — Call Gemini and Parse Response").

An earlier prompt version described the evidence field inconsistently. On "Attention Is All You Need", Gemini resolved the ambiguity by returning evidence as one flat string with the heading prepended, e.g. "## Introduction In this work we propose...", instead of the nested {section, quote} object. I first fixed this by rewriting the prompt; the schema now guarantees the nested shape regardless of prompt wording. `response_schema` and `response_json_schema` are not interchangeable. `response_schema=MethodologyProfile` fails with `400 INVALID_ARGUMENT ... Unknown name "additional_properties"`, because it converts to Google's own Schema proto, which does not support `additionalProperties`, and Pydantic's `extra="forbid"` produces exactly that field. `response_json_schema` accepts a real JSON Schema dict instead, so `MethodologyProfile.model_json_schema()` is passed there.

Third, the Variant B prompts share one skeleton and the same two rules (null when absent, verbatim quotes), and only one rule line changes per role. The difference between Variant A and Variant B is therefore the decomposition plus that one line, which keeps the comparison controlled.

Fourth, the scoring semantics. Because `matches` is a substring test, the gold label "F1" matches "F1 score", while the gold Task "GLUE" does not match "language representation". Treating a wrong non-null answer as both a false positive and a false negative means precision penalises confident wrong answers and recall penalises abstention, which suits a tool whose output users are meant to trust. For code quality, `pyright` in strict mode and `ruff` report zero issues, and a

pytest suite in proto3/tests/ covers scoring.py, the run aggregation, and the validator (make lint, make test). proto3/baseline.ipynb used to be a byte-identical duplicate of 3pipeline.ipynb, kept only because it had produced the six proto3/baseline/*.json files; it has since been deleted, so there is no second notebook to keep in sync by hand.

4. Visual Representation

For "Attention Is All You Need" [D6], the full extraction output is:

```
{
  "TechnicalMethod": {
    "answer": "Transformer",
    "evidence": {
      "section": "Abstract",
      "quote": "We propose a new simple network
architecture, the Transformer, based solely on
attention mechanisms, dispensing with recurrence
and convolutions entirely."
    }
  },
  "Task": {
    "answer": "machine translation",
    "evidence": {
      "section": "Abstract",
      "quote": "Experiments on two machine
translation tasks show these models to be
superior in quality while being more
parallelizable and requiring significantly less
time to train."
    }
  },
  "Dataset": {
    "answer": "WMT 2014 English-German",
    "evidence": {
      "section": "Training Data and Batching",
      "quote": "We trained on the standard WMT
2014 English-German dataset consisting of about
4.5 million sentence pairs."
    }
  }
}
```

```

    }
  },
  "EvaluationMetric": {
    "answer": "BLEU",
    "evidence": {
      "section": "Machine Translation",
      "quote":
        "On the WMT 2014 English-to-German translation
        task, the big transformer model (Transformer
        (big) in Table 2) outperforms the best previously
        reported models (including ensembles) by more
        than 2.0 BLEU, establishing a new state-of-the-
        art BLEU score of 28.4."
    }
  }
}

```

Figure 7: Full extraction output for "Attention Is All You Need" [D6] (`proto3/baseline/transformer.json`).

Figure 8 shows the Stage 2c cell with the raw Gemini call and its parsed JSON output, so the extraction is visible running directly.

```

[12] ✓ 9s
response = client.models.generate_content(
    model=MODEL_NAME,
    contents=prompt,
    config=types.GenerateContentConfig(
        temperature=0,
        seed=0,
        response_mime_type="application/json",
        response_json_schema=MethodologyProfile.model_json_schema(),
    ),

    profile = MethodologyProfile.model_validate_json(response.text)

    for role in ROLES:
        print(f"{role}: {getattr(profile, role)}")

    print()
    print(profile.model_dump_json(indent=2))

...
TechnicalMethod: answer='Transformer' evidence=Evidence(section='Abstract', quote='We propose a new simple network
Task: answer='machine translation' evidence=Evidence(section='Abstract', quote='Experiments on two machine transla
Dataset: answer='WMT 2014 English-German dataset' evidence=Evidence(section='Training Data and Batching', quote='W
EvaluationMetric: answer='BLEU' evidence=Evidence(section='Machine Translation', quote='On the WMT 2014 English-to

{
  "TechnicalMethod": {
    "answer": "Transformer",
    "evidence": {
      "section": "Abstract",
      "quote": "We propose a new simple network architecture, the Transformer, based solely on attention mechanism
    }
  },
  "Task": {
    "answer": "machine translation",
    "evidence": {
      "section": "Abstract",
      "quote": "Experiments on two machine translation tasks show these models to be superior in quality while be
    }
  },
  "Dataset": {
    "answer": "WMT 2014 English-German dataset",
    "evidence": {
      "section": "Training Data and Batching",
      "quote": "We trained on the standard WMT 2014 English-German dataset consisting of about 4.5 million sentenc
    }
  },
  "EvaluationMetric": {
    "answer": "BLEU",
    "evidence": {
      "section": "Machine Translation",

```

Figure 8: proto3 Stage 2c cell and output (screenshot).

Table 8 shows the contrast with proto2 numerically for the Transformer paper: proto2's accepted-sentence counts per role versus proto3's one answer per role.

Paper	proto2 Technical Method	proto2 Task	proto2 Dataset	proto2 EvaluationMetric	proto3
Transformer [D6]	14 sentences	0 sentences	0 sentences	160 sentences	1 answer + evidence per role

Table 8: proto2 sentence-count output vs proto3 answer-and-evidence output, Transformer paper.

Table 9 compares the Transformer output of Variant A and Variant B. Only Task differs, and the Variant B answer comes from a Conclusion sentence: "In this work, we presented the Transformer, the first sequence transduction model based entirely on attention..."

Role	Variant A	Variant B
Technical Method	Transformer	Transformer
Task	machine translation	sequence transduction
Dataset	WMT 2014 English-German dataset	WMT 2014 English-German dataset
Evaluation Metric	BLEU	BLEU

Table 9: Variant A (proto3/results/run1) and Variant B (proto3/results_b) answers for the Transformer paper.

5. Evaluation (2234/2500 words)

1. Evaluation Method

I evaluate gold-label match as a classification problem — Precision, Recall, and F1 per role, with Wilson 95% confidence intervals on Precision and Recall — backed by a logged 5-run variance study and a consolidated manual review pass covering plausibility, evidence support, authorship, and whether the quote appears in the source text, all in a single read. Variant B and the Task pilots are scored against the same gold labels.

Precision/Recall/F1 (P/R/F1) is more appropriate than proto2's recall-only substring check, because a present-but-wrong answer now costs both precision and recall, instead of being free the way it was when

any accepted sentence containing the gold term counted as a hit, regardless of how many other sentences were also returned.

The manual review folds the evidence-verbatim check into the same pass as support and authorship, rather than a separate automated script: a verbatim match only proves the LLM followed the copy-verbatim instruction, not that the evidence is good evidence (a real, verbatim quote from a Related Work sentence could still be the wrong evidence for a paper's own methodology), and the reviewer already has to read the source to judge support and authorship.

2. Gold-Label-Match Results

Scoring the frozen baseline (proto3/baseline/*.json) against gold labels (Appendix A, Table A1) across all six papers gives:

Role	P	R	F1
TechnicalMethod	0.83	0.83	0.83
Task	0.33	0.33	0.33
Dataset	0.80	0.67	0.73
EvaluationMetric	0.80	0.67	0.73
Micro	0.68	0.62	0.65
Macro	—	—	0.655

Table 10: Baseline gold-label-match results, all six papers.

I report macro as the headline "Overall" score, because the four roles are fixed, equally mandatory schema fields, not a frequency distribution — a user needs all four, not just whichever role happens to have the most examples. The two averages are close here (0.65 vs 0.655) only because every role happens to have $n=6$ in this dataset, a property of the dataset and not of the method.

Wilson 95% confidence intervals show the effect of the sample size: TechnicalMethod recall 0.83 gives a confidence interval of [0.44, 0.97]; Task recall 0.33 gives [0.10, 0.70]. These substantially overlap,

so I do not claim TechnicalMethod is reliably "solved" while Task is reliably "broken" at this sample size. I report F1 as a point estimate, without a Wilson interval (Chapter 3 §5). One gold label carries a specific evaluator-influence caveat: AlexNet's TechnicalMethod gold label was changed from "AlexNet" to "convolutional" after running the pipeline and inspecting its output, since the 2012 paper predates the name "AlexNet" and never uses it. Adjusting a gold label after seeing model output limits how far this result generalises, and it is one instance of a broader single-annotator problem: I wrote both the gold labels and, later, the answers checked against them (Section 4 below).

Figure 9 shows the aggregate baseline scoring running in the notebook, and Figure 10 a single-paper breakdown (Transformer), baseline versus pipeline.

The screenshot shows a Google Colab notebook interface. The top bar displays the URL: `https://colab.research.google.com/github/sanemat/uol-fp/blob/main/proto3/3pipeline.ipynb#scrollTo...`. The notebook has tabs for 'Commands', 'Code', 'Text', 'Run all', and 'Save in GitHub to keep changes'. The 'Code' tab is active, showing a Python script. The script defines a `print_score_table` function that takes a dictionary of totals and a title, and prints a table of scores. It then calls this function with `BASELINE_TOTALS` and a title. Below the code, the output of the function is displayed as a table. The table has columns for Role, TP, FP, FN, TN, P, R, and F1. The rows represent different roles: TechnicalMethod, Task, Dataset, EvaluationMetric, and Overall. The 'Overall' row shows the aggregated scores: TP=15, FP=7, FN=9, TN=0, P=0.68, R=0.62, F1=0.65. Below the table, there is a code cell that defines `pipeline_answers`, `gold_answers`, and `baseline_answers`, and prints a comparison of the scores.

```

return totals

def print_score_table(totals: dict[str, tuple[int, int, int, int]], title: str) -> None:
    print(title)
    print(
        f"{'Role':<18} {'TP':>3} {'FP':>3} {'FN':>3} {'TN':>3}"
        f" {'P':>5.2f} {'R':>5.2f} {'F1':>5.2f}"
    )
    overall = (0, 0, 0, 0)
    for role in ROLES:
        tp, fp, fn, tn = totals[role]
        p, r, f1 = precision_recall_f1(tp, fp, fn)
        print(
            f"{'role':<18} {'tp':>3} {'fp':>3} {'fn':>3} {'tn':>3}"
            f" {'p':>5.2f} {'r':>5.2f} {'f1':>5.2f}"
        )
        overall = tuple(a + b for a, b in zip(overall, (tp, fp, fn, tn)))
    p, r, f1 = precision_recall_f1(overall[0], overall[1], overall[2])
    print(
        f"{'Overall':<18} {'overall[0]:>3} {'overall[1]:>3} {'overall[2]:>3}"
        f" {'overall[3]:>3}"
        f" {'p':>5.2f} {'r':>5.2f} {'f1':>5.2f}"
    )
    print()

BASELINE_TOTALS = aggregate_scores(BASELINE_ANSWERS)
print_score_table(BASELINE_TOTALS, "Baseline (frozen proto3/baseline/*.json) vs gold:")

... Baseline (frozen proto3/baseline/*.json) vs gold:
Role      TP  FP  FN  TN    P    R    F1
TechnicalMethod  5   1   1   0  0.83 0.83 0.83
Task        2   4   4   0  0.33 0.33 0.33
Dataset     4   1   2   0  0.80 0.67 0.73
EvaluationMetric 4   1   2   0  0.80 0.67 0.73
Overall    15   7   9   0  0.68 0.62 0.65

[ ]
19s ▶ pipeline_answers = {role: getattr(profile, role).answer for role in ROLES}
gold_answers = GOLD_LABELS[PAPER_SLUG]
baseline_answers = BASELINE_ANSWERS[PAPER_SLUG]

print(f"{'Role':<18} {'Gold':<25} {'Baseline':<25} {'Pipeline':<25}")
for role in ROLES:
    gold = gold_answers.get(role) or "(none)"
    baseline = baseline_answers.get(role) or "(none)"
    pipeline = pipeline_answers.get(role) or "(none)"
    print(f"{'role':<18} {'gold':<25} {'baseline':<25} {'pipeline':<25}")
print()

print_score_table(

```

Variables Terminal 9:32 AM Python 3

Figure 9: Baseline P/R/F1 scoring output
(`proto3/3pipeline.ipynb`, Stage 3).

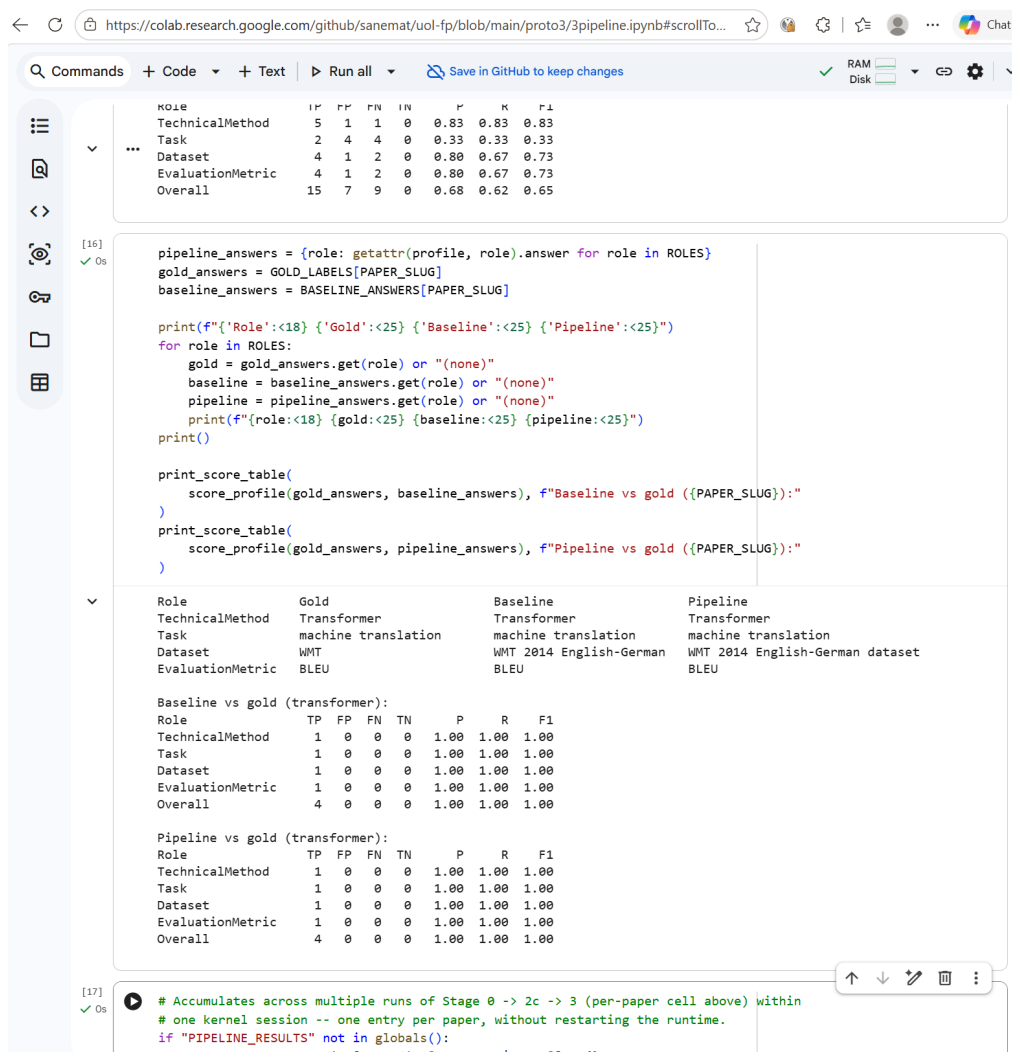


Figure 10: Per-paper gold-label scoring for the Transformer paper, baseline vs. pipeline (`proto3/3pipeline.ipynb`, Stage 3).

3. Variance Study

I logged five full pipeline runs to `proto3/results/run{1..5}/*.json` and aggregated them with `proto3/aggregate_runs.py`. Per-role F1 across the five runs:

Role	F1 mean	F1 min	F1 max	F1 range
TechnicalMethod	0.83	0.83	0.83	0.00
Task	0.33	0.33	0.33	0.00
Dataset	0.91	0.91	0.91	0.00

Role	F1 mean	F1 min	F1 max	F1 range
EvaluationMetric	0.57	0.33	0.67	0.33

Table 11: Per-role F1 across 5 real pipeline runs (proto3/results/aggregate.json).

Three of four roles were perfectly stable across five real repetitions, which is stronger evidence than an earlier two-run anecdote in which both Dataset and EvaluationMetric had moved. At $n=5$, only EvaluationMetric varied (F1 ranged 0.33-0.67 across runs, with unchanged code, temperature=0, and seed=0), which narrows the non-determinism finding. The frozen baseline behind Table 10 is not a like-for-like sixth run alongside these five: it was generated before temperature=0 and seed=0 were added to the Gemini call, so only the five logged runs share identical settings. Across the five runs, TechnicalMethod's F1 (0.83) exceeded Task's F1 (0.33) every time.

An informal cross-check with Google NotebookLM, run independently on each paper, found stable agreement on TechnicalMethod across all six papers — exact or near-exact matches including "Google", "BERT", "Transformer", and "MapReduce". This is independent corroboration that TechnicalMethod is the strongest role.

I did not pool the five runs into a single Wilson interval (Chapter 3 §5). Paper-level uncertainty, meaning how results might vary across a different sample of papers, is covered by the $n=6$ baseline intervals in Section 2. The five-run study answers a different question: how the same six papers' results vary when the pipeline is simply run again. Table 11 answers it with mean, minimum, maximum, and range, and the consistent gap between TechnicalMethod and Task needs no interval to state.

MapReduce's Task slot (gold "distributed", system answer "automatic parallelization and distribution of large-scale computations") fails the substring-match rule despite being arguably correct. Task's low F1 is therefore partly an artifact of the measurement instrument and not purely a model failure.

MapReduce's Dataset slot (gold "TeraSort") answered null in every one of the five runs, a genuine recall miss, since NotebookLM independently found the dataset description (two roughly 1 terabyte grep/sort benchmarks) in the same source text. Pagerank's EvaluationMetric slot (gold "quality") answered "precision" in every one of the five runs. It is scored wrong by substring match, but the paper's text supports both terms, and NotebookLM's independent extraction also names precision.

4. Manual Review

The review template stages each paper's answer, section, and quote next to four judgment columns (plausible? evidence supports? authors' own work? quote in source?). The template also staged two open questions before scoring. BERT's Task quote cites prior work by name, so the review had to decide whether pre-training is BERT's own contribution or only motivation. ResNet's Task quote cites bracketed prior work for "a series of breakthroughs," which raises the same question of whether the sentence establishes the paper's own task or credits others. All 24 slots are scored. Two are null (Pagerank/EvaluationMetric, MapReduce/Dataset), judged separately below; across the other 22 scored slots:

Check	Failures (of 22)	Slots
Plausible?	2	Pagerank/TechnicalMethod, Pagerank/Task
Evidence supports?	5	Pagerank/TechnicalMethod, Pagerank/Task, AlexNet/Task, BERT/Task, BERT/Dataset
Authors' own work?	6	Pagerank/Task, AlexNet/Task, BERT/Task, BERT/Dataset, BERT/EvaluationMetric, ResNet/Task
Quote in source?	1	ResNet/Task

Table 12: Manual review failure counts by check type, 22 scored slots.

Six slots pass the quote-in-source check but still fail on evidence-support or authorship — a real, verbatim quote that is still the *wrong* evidence, distinct from a fabricated quote: Pagerank/TechnicalMethod ("Google is the proposed system, not the technical method"), Pagerank/Task ("information retrieval is the broader problem domain, not the task performed by the proposed system"), AlexNet/Task ("the quote does not directly prove that this is AlexNet's own task"), BERT/Task (the quoted sentence "describes prior work rather than BERT's own contribution"), BERT/Dataset ("valid dataset, but one of several"), and BERT/EvaluationMetric ("F1 is one of several evaluation metrics used across BERT's downstream tasks"). ResNet's Task answers the second open question above and is the one slot where quote-in-source and authorship fail together: the quote "cites bracketed prior work [21, 49, 39] for the breakthroughs," crediting prior work rather than establishing the paper's own task.

On the two null slots: MapReduce's Dataset is a real miss, not a genuine absence — the paper "uses large benchmark datasets for grep and sort experiments (about 1 TB each)," which NotebookLM independently found in the source text. Pagerank's EvaluationMetric leans toward a gold-label problem: precision "is discussed as an important search-quality criterion, but it is not actually measured or reported as an experimental metric" — gold "quality" is likely too vague or mislabeled.

BERT's Dataset and EvaluationMetric failures confirm the multi-valued-roles finding from Chapter 3: choosing only one of several correct answers can be technically correct but still gives an incomplete picture. Some gold answers may also be too simple or use the wrong category, so evaluating against gold labels alone can give a misleading result. The review has the same single-annotator bias as the gold labels it checks against: I wrote both the gold labels and, later, the answers checked against them, so I do not present this review as more objective than the gold labels themselves.

5. Decomposed Extraction and the Task Pilots

Variant B (four role-specific calls) ran once on all six papers and was scored against run 1 of Variant A, so both columns are single runs:

Role	A F1	B F1	$\Delta F1$
TechnicalMethod	0.83	0.83	+0.00
Task	0.33	0.33	+0.00
Dataset	0.91	1.00	+0.09
EvaluationMetric	0.67	0.67	+0.00

Table 13: Variant A (results/run1) vs Variant B (results_b), F1 per role, one run each.

Only Dataset moved (+0.09), and its prompt was not written against a known Dataset failure. Task's F1 stayed at 0.33, but the two matching papers differ. For Pagerank, Variant B replaced an Introduction sentence about the Web's challenges for information retrieval (rated wrong in the manual review) with "web search", taken from the Abstract's statement of what the paper addresses, and the substring test now matches. For Transformer, Variant B answered "sequence transduction" from a Conclusion sentence describing the architecture, where Variant A had answered "machine translation" from the Abstract's evaluated task (Table 9). The role-specific rule separates the paper's own claim from prior work, but it cannot prefer the evaluated benchmark over a broader self-description, and the two changes cancel. Variant C only reconciles Variant B's four outputs with each other, so it could not resolve this ambiguity, and I did not run it.

Five further mechanisms targeted Task on the same six papers, each run once (Table 14). Stage 2e asked for every valid Task description, most specific first. Stage 2f asked a second call to select one candidate. Stage 4 replaced the substring match with a Gemini judge on Variant A's four mismatches. Stages 2g and 2h asked the model to write its reasoning before its answer, either to select from Stage 2e's candidates (2g) or to extract directly (2h), adapting Lu et al. [12].

Mechanism	P	R	F1	Note
Variant A (joint)	0. 33	0. 33	0.33	2 TP, 4 FP, 4 FN
Variant B (decomposed)	0. 33	0. 33	0.33	different two papers match
Stage 2e, any- match	0. 67	0. 67	0.67	match anywhere in a list of 2-7 items
Stage 2e, primary answer only	0. 17	0. 17	0.17	only BERT's first item matched
Stage 2f, verification	0. 33	0. 33	0.33	BERT's correct answer replaced
Stage 4, LLM judge	—	—	0.67 (about 0.50 after review)	one of two SAME verdicts contradicts manual review
Stage 2g, reasoning-first selection	0. 50	0. 33	0.40	gain from returning null on BERT and MapReduce
Stage 2h, reasoning-first extraction	0. 33	0. 33	0.33	same hits as Variant A

Table 14: Task F1 under each mechanism, six papers, one run each (proto3/results_*).

The any-match score doubles F1 but is bounded only by the length of the candidate list. Of its four hits, only BERT's was the model's first choice; the others were listed second to fourth in lists of 2-5 items. Stage 2f and Stage 2g both removed BERT's answer "GLUE benchmark" (correct as Stage 2e's first item), replacing it with a wrong answer and with null respectively. Stage 2g's 0.40 comes entirely from abstention, since a null against a non-null gold label counts as a false negative only. The Stage 2g reasoning traces evaluate each candidate against an explicit granularity test, so the

model reasons coherently and still lands on a different phrasing from the gold label or declines to answer.

The Stage 4 judge returned SAME for MapReduce's "automatic parallelization and distribution of large-scale computations" against "distributed", which agrees with the earlier reading in Section 3. It also returned SAME for Pagerank's "information retrieval" against "web search", which contradicts the manual review of the same answer (Section 4). The judge and the extractor are the same model family, and with four judged pairs I could not establish whether its verdicts can be trusted in general. A lexical alternative also fails on these pairs: Jaccard similarity between the gold label and the answer is 0.00-0.25, far below the 0.8 threshold that Ateia et al. [1] use.

These results agree with published findings. Tam et al. [17] report that restricting LLM output to a format can lower performance, which is compatible with the gap between Stage 2e's any-match (0.67) and primary-only (0.17) scores. Huang et al. [7] report that LLMs cannot reliably correct their own reasoning without external feedback, which is compatible with the regressions in Stages 2f, 2g, and 4. Ateia et al. [1] score free-text extraction targets with BERTScore F1 [19] and report a semantic F1 near 0.90 on those targets, while exact categorical extraction stays below 0.25. The two scores cover different target types, but they show that a lenient semantic score and a strict match can differ widely. Task's residual F1 may reflect the single-gold-phrasing substring metric as much as the model. A deterministic semantic-similarity rescoring is planned but not run; it would need one threshold that accepts the MapReduce pair and rejects the Pagerank pair.

Two limitations apply to the Task line. The Stage 2f prompt was revised while inspecting the Transformer failure, and its first version used two of the project's own gold Task labels as examples and was discarded before any data collection. Every mechanism is a single run on six papers, so differences of one paper (0.17 in F1) are within the run-to-run variation seen for EvaluationMetric in Table 11.

I did not run the Related Work ablation, which would exclude the Related Work section before extraction. It does not address Task, the weakest role, and the core claims do not depend on it, so it stays deferred to further work (Chapter 6).

6. Critical Evaluation

Mapping proto2's three named failure modes, and the Task weakness found later, onto what the evaluation measured across the whole project:

Failure mode	Status	Evidence
Output volume (151 TechnicalMethod sentences for MapReduce)	Fixed by design	One answer per role, every paper, by construction of Stage 2's schema-guided extraction
No authorship-attribution mechanism (ELMo scored 0.87 as BERT's TechnicalMethod)	Addressed by design, not reliably solved	The authors'-own-work rule targets this directly; the manual review found 6 of 22 scored slots still fail authorship, concentrated in Task (4 of 6 papers)
Recall-only evaluation (10/12, then 18/24 substring match)	Fixed	Precision/Recall/F1 per role, Wilson confidence intervals on Precision/Recall, a 5-run variance study
Task F1 of 0.33	Not fixed	Six mechanisms (Table 14) did not raise it above 0.33 under a check I could trust

Table 15: Failure modes across proto2, proto3, and the Task experiments.

Successes. The move from proto2's recall-only substring check to Precision/Recall/F1 with confidence intervals, backed by five repeated

runs, gives a measurement that a present-but-wrong answer can fail. Every answer carries evidence that a reader can check. TechnicalMethod (F1 0.83) and Dataset (0.91) are stable across five runs, and TechnicalMethod exceeded Task in every run.

Failures. Task's F1 is 0.33 in all five runs and under Variants A and B, and none of the six mechanisms in Section 5 improved it.

MapReduce's Dataset slot answered null in all five runs (gold "TeraSort"), a model recall failure confirmed by the NotebookLM cross-check, which is separate from the gold-label problems elsewhere. Six of 22 scored slots fail the authorship check, so the authors'-own-work rule reduces but does not remove the citation problem that motivated it.

Limitations. The corpus has six papers, and one annotator wrote both the gold labels and the review. AlexNet's gold label was changed after seeing the output. The four-role schema comes from SciREX's ML-conference corpus (Chapter 2, Section 2); two of the six papers are systems papers (MapReduce, Google Search), and proto2 already showed that these fit the schema worse than the four ML papers. Variant B and the Task pilots are single runs. I tested one general-purpose top model (Gemini), so the results say nothing about whether another model performs better. The evaluation involved no user study, so the claim that the output supports the first pass of a literature review rests on the evidence checks in Section 4 and not on measured reader behaviour.

Extensions. The next steps are a deterministic semantic-match rescoring of Task, Variant C, a multi-valued schema for Dataset and EvaluationMetric, more papers with a second annotator, and papers outside ML benchmarks (Chapter 6).

6. Conclusion (680/1000 words)

Summary

This project automatically extracts research methodology — technical method, task, dataset, and evaluation metric — from computing

research papers using large language models. I built two prototype iterations: proto2, my own sentence-level zero-shot NLI classifier, which produced too much unusable output and could not separate authors' own methods from cited prior work; and proto3, a document-level, schema-guided extraction pipeline that returns one evidence-backed answer per role per paper. On top of proto3 I implemented a decomposed variant and six mechanisms aimed at the weak Task role.

Did it work? Partially. TechnicalMethod and Dataset extraction reach F1 0.83 and 0.91 with stable results across five runs, and the output is far more usable than proto2's. Task stays at F1 0.33 under every mechanism I tried, and the schema's fit to systems papers remains open.

What did the project teach? Producing structured JSON was not the hard part; `response_json_schema` solves schema conformance (Chapter 4). The hard part was deciding what the four roles mean consistently across different kinds of computing papers. MapReduce and Google Search struggle partly because Task, Dataset, and EvaluationMetric come from ML-benchmark structure and do not map cleanly onto systems research. The Task experiments point the same way: the model's reasoning about Task is coherent, and the gold label's single phrasing is one defensible choice among several that the substring metric cannot distinguish.

Did it meet the original user need? Chapter 1 set the goal of supporting the first pass of a literature review. proto2's output was too voluminous for that purpose; proto3's one-answer-per-role-plus-evidence format is a clear improvement. Task's low F1 and the single-valued Dataset and EvaluationMetric roles mean a user still needs to verify results by hand.

Further Work

The corpus stayed at six papers, which limits generalisability (Chapter 3 §5). Growing it to the 30-40 gold-labeled papers per role needed for narrower confidence intervals, with a second human annotator, would

allow a formal inter-annotator-agreement study. The informal NotebookLM cross-check (Chapter 5) narrows this gap but does not close it, since it is one AI tool's single pass without an annotation protocol.

The Task line has a specific next step. Stage 5 would rescore all Task variants with a deterministic semantic similarity (BERTScore [19] or embedding cosine) and test whether one threshold accepts the MapReduce pair and rejects the Pagerank pair; if none does, the semantic metric has the same leniency problem as the LLM judge. Variant C, a fifth call that checks Variant B's four outputs against their evidence, remains untested. My hypothesis is that per-role accuracy follows $B \text{ or } C > A$ and cross-role consistency follows $C > A > B$, but Variant B's result gives little reason to expect a gain on Task.

The multi-valued schema for Dataset and EvaluationMetric also remains to be implemented: a MultiRoleExtraction type with per-item evidence and a ranked list capped at three items, gold-label re-annotation for the four cells already identified (BERT and Transformer Dataset, AlexNet and ResNet EvaluationMetric), a parallel `score_role_multi` scoring function, and rerunning and rescoring all six papers. A comparison of model quality, cost, and reproducibility across alternative models would show whether a more suitable model exists. It only becomes relevant now that one general-purpose top model has shown the approach is useful. The Related Work ablation would test whether excluding sentences about prior work reduces the authorship failures found in the manual review. Testing on non-ML-benchmark papers such as systems and HCI would show whether the four-role schema generalises, and a user study with computing students would measure whether the profiles support the first pass of a literature review.

Broader Theme

`response_json_schema` guarantees that Gemini's reply is syntactically valid and has the right shape. It does not guarantee that the content is correct: an answer can be well-formed and still wrong, as the manual review in Chapter 5 shows. The Task experiments add a third

case, where an answer is well-formed, well-reasoned, and marked wrong by the metric because it uses a different phrasing from the gold label. The distinction between schema conformance and semantic correctness applies to LLM-based structured extraction generally.

References

- [1] Samy Ateia, Udo Kruschwitz, Melanie Scholz, Agnes Koschmider, and Moayad Almohaishi. 2025. LLM-Based Information Extraction to Support Scientific Literature Research and Publication Workflows. In *New Trends in Theory and Practice of Digital Libraries (TPDL 2025)*. Springer Nature Switzerland, 90–99. https://doi.org/10.1007/978-3-032-06136-2_9
- [2] John Dagdelen, Alexander Dunn, Sanghoon Lee, Nicholas Walker, Andrew S. Rosen, Gerbrand Ceder, Kristin A. Persson, and Anubhav Jain. 2024. Structured information extraction from scientific text with large language models. *Nature Communications* 15 (2024), 1418. <https://doi.org/10.1038/s41467-024-45563-x>
- [3] Michael Färber, Alexander Albers, and Felix Schüber. 2021. Identifying Used Methods and Datasets in Scientific Publications. In *Proceedings of the Second Workshop on Scholarly Document Understanding (SDU@AAAI 2021)*. <https://ceur-ws.org/Vol-2831/paper19.pdf>
- [4] Madhusudan Ghosh, Debasis Ganguly, Partha Basuchowdhuri, and Sudip Kumar Naskar. 2023a. Enhancing AI research paper analysis: methodology component extraction using factored transformer-based sequence modeling. arXiv:2311.03401. <https://doi.org/10.48550/arXiv.2311.03401>
- [5] Madhusudan Ghosh, Debasis Ganguly, Partha Basuchowdhuri, and Sudip Kumar Naskar. 2023b. Extracting methodology components from AI research papers: a data-driven factored sequence labeling approach. In *Proceedings of the 32nd ACM International Conference*

on Information and Knowledge Management (CIKM 2023). 3897–3901.
<https://doi.org/10.1145/3583780.3615258>

[6] Pengcheng He, Jianfeng Gao, and Weizhu Chen. 2021. DeBERTaV3: Improving DeBERTa using ELECTRA-Style Pre-Training with Gradient-Disentangled Embedding Sharing. arXiv:2111.09543. <https://doi.org/10.48550/arXiv.2111.09543>

[7] Jie Huang, Xinyun Chen, Swaroop Mishra, Huaixiu Steven Zheng, Adams Wei Yu, Xinying Song, and Denny Zhou. 2024. Large Language Models Cannot Self-Correct Reasoning Yet. In *The Twelfth International Conference on Learning Representations (ICLR 2024)*. arXiv: 2310.01798. <https://doi.org/10.48550/arXiv.2310.01798>

[8] Sarthak Jain, Madeleine van Zuylen, Hannaneh Hajishirzi, and Iz Beltagy. 2020. SciREX: A Challenge Dataset for Document-Level Information Extraction. In *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, Online, July 2020. Association for Computational Linguistics, 7506–7516. <https://doi.org/10.18653/v1/2020.acl-main.670>

[9] Tushar Khot, Harsh Trivedi, Matthew Finlayson, Yao Fu, Kyle Richardson, Peter Clark, and Ashish Sabharwal. 2022. Decomposed Prompting: A Modular Approach for Solving Complex Tasks. arXiv: 2210.02406. <https://doi.org/10.48550/arXiv.2210.02406>

[10] Zsolt T. Kosztyán, Tünde Király, Tibor Csizmadia, Attila Imre Katona, and Ágnes Vathy-Fogarassy. 2025. Automated research methodology classification using machine learning. *Engineering Applications of Artificial Intelligence* 156 (2025), article 111039. <https://doi.org/10.1016/j.engappai.2025.111039>

[11] Patrice Lopez. 2009. GROBID: Combining Automatic Bibliographic Data Recognition and Term Extraction for Scholarship Publications. In *Research and Advanced Technology for Digital Libraries: Proceedings of ECDL 2009*. Springer Berlin Heidelberg, Berlin, Heidelberg, 473–474. https://doi.org/10.1007/978-3-642-04346-8_62

- [12] Yaxi Lu, Haolun Li, Xin Cong, Zhong Zhang, Yesai Wu, Yankai Lin, Zhiyuan Liu, Fangming Liu, and Maosong Sun. 2025. Learning to Generate Structured Output with Schema Reinforcement Learning. arXiv:2502.18878. <https://doi.org/10.48550/arXiv.2502.18878>
- [13] Yongqiang Ma, Jiawei Liu, Wei Lu, and Qikai Cheng. 2023. From "what" to "how": Extracting the procedural scientific information toward the metric-optimization in AI. *Information Processing & Management*, 60(3), article 103315. <https://doi.org/10.1016/j.ipm.2023.103315>
- [14] Briony J. Oates. 2006. *Researching Information Systems and Computing*. SAGE Publications, London.
- [15] Colin Pilkington and Laurette Pretorius. 2015. A conceptual model of the research methodology domain. In *Proceedings of the 7th International Joint Conference on Knowledge Discovery, Knowledge Engineering and Knowledge Management (IC3K 2015), Volume 2: KEOD*. SCITEPRESS – Science and Technology Publications, Setúbal, Portugal, 96–107. <https://doi.org/10.5220/0005613100960107>
- [16] Maciej P. Polak and Dane Morgan. 2024. Extracting accurate materials data from research papers with conversational language models and prompt engineering. *Nature Communications* 15 (2024), 1569. <https://doi.org/10.1038/s41467-024-45914-8>
- [17] Zhi Rui Tam, Cheng-Kuang Wu, Yi-Lin Tsai, Chieh-Yen Lin, Hung-yi Lee, and Yun-Nung Chen. 2024. Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing: Industry Track*. Association for Computational Linguistics, 1218–1236. <https://doi.org/10.18653/v1/2024.emnlp-industry.91>
- [18] Wenpeng Yin, Jamaal Hay, and Dan Roth. 2019. Benchmarking Zero-shot Text Classification: Datasets, Evaluation and Entailment Approach. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*,

Hong Kong, China, November 2019. Association for Computational Linguistics, 3912–3921. <https://doi.org/10.18653/v1/D19-1404>

[19] Tianyi Zhang, Varsha Kishore, Felix Wu, Kilian Q. Weinberger, and Yoav Artzi. 2020. BERTScore: Evaluating Text Generation with BERT. In *International Conference on Learning Representations (ICLR 2020)*. arXiv:1904.09675. <https://doi.org/10.48550/arXiv.1904.09675>

Dataset Papers

[D1] Sergey Brin and Lawrence Page. 1998. The anatomy of a large-scale hypertextual web search engine. *Computer Networks and ISDN Systems*, 30(1–7), 107–117. [https://doi.org/10.1016/S0169-7552\(98\)00110-X](https://doi.org/10.1016/S0169-7552(98)00110-X)

[D2] Jeffrey Dean and Sanjay Ghemawat. 2004. MapReduce: Simplified data processing on large clusters. In *Proceedings of the 6th Symposium on Operating Systems Design and Implementation (OSDI '04)*. USENIX Association, 137–150. <https://www.usenix.org/conference/osdi-04/mapreduce-simplified-data-processing-large-clusters>

[D3] Jacob Devlin, Ming-Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT 2019)*, Volume 1. Minneapolis, Minnesota: Association for Computational Linguistics, 4171–4186. <https://doi.org/10.18653/v1/N19-1423>

[D4] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR 2016)*, 770–778. <https://doi.org/10.1109/CVPR.2016.90>

[D5] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems*, 25, 1097–1105.

https://proceedings.neurips.cc/paper_files/paper/2012/hash/c399862d3b9d6b76c8436e924a68c45b-Abstract.html

[D6] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, 30, 5998–6008. <https://proceedings.neurips.cc/paper/2017/hash/3f5ee243547dee91fbd053c1c4a845aa-Abstract.html>

Appendix A — Extended proto3 Output and Background Data (All 6 Papers)

Chapter 4 shows the full extraction output for the Transformer paper (Figure 7). This appendix gives the same output for the remaining five papers, all from `proto3/baseline/*.json`, plus the gold labels and `proto2` background counts used throughout Chapters 4–5.

```
{
  "TechnicalMethod": {
    "answer": "deep convolutional neural network",
    "evidence": {
      "section": "Abstract",
      "quote": "We trained a large, deep convolutional
neural network to classify the 1.2 million high-resolution
images in the ImageNet LSVRC-2010 contest into the 1000
different classes."
    }
  },
  "Task": {
    "answer": "object recognition",
    "evidence": {
      "section": "Introduction",
      "quote": "Current approaches to object recognition
make essential use of machine learning methods."
    }
  },
  "Dataset": {
    "answer": "ImageNet",
```

```

    "evidence": {
      "section": "The Dataset",
      "quote": "ImageNet is a dataset of over 15 million
labeled high-resolution images belonging to roughly 22,000
categories."
    },
    "EvaluationMetric": {
      "answer": "top-1 and top-5 error rates",
      "evidence": {
        "section": "The Dataset",
        "quote":
"On ImageNet, it is customary to report two error rates:
top-1 and top-5, where the top-5 error rate is the
fraction of test images for which the correct label is not
among the five labels considered most probable by the
model."
      }
    }
  }
}

```

Figure A1: Full extraction output for AlexNet [D5].

```

{
  "TechnicalMethod": {
    "answer": "BERT",
    "evidence": {
      "section": "Abstract",
      "quote":
"We introduce a new language representation model called
BERT, which stands for Bidirectional Encoder
Representations from Transformers."
    }
  },
  "Task": {
    "answer": "Language model pre-training",
    "evidence": {
      "section": "Introduction",
      "quote":
"Language model pre-training has been shown to be
effective for improving many natural language processing

```

```

tasks (Dai and Le, 2015; Peters et al., 2018a; Radford et
al., 2018; Howard and Ruder, 2018) ."
    },
    "Dataset": {
        "answer": "SQuAD v1.1",
        "evidence": {
            "section": "SQuAD v1.1",
            "quote": "The Stanford Question Answering Dataset
(SQuAD v1.1) is a collection of 100k crowdsourced
question/answer pairs (Rajpurkar et al., 2016) ."
        }
    },
    "EvaluationMetric": {
        "answer": "F1 score",
        "evidence": {
            "section": "SQuAD v1.1",
            "quote": "Our single BERT model outperforms the top
ensemble system in terms of F1 score."
        }
    }
}

```

Figure A2: Full extraction output for BERT [D3].

```

{
    "TechnicalMethod": {
        "answer": "MapReduce",
        "evidence": {
            "section": "Abstract",
            "quote": "MapReduce is a programming model and an
associated implementation for processing and generating
large data sets."
        }
    },
    "Task": {
        "answer": "automatic parallelization and distribution
of large-scale computations",
        "evidence": {
            "section": "Introduction",
            "quote":

```

```

    "The major contributions of this work are a simple and
    powerful interface that enables automatic parallelization
    and distribution of large-scale computations, combined
    with an implementation of this interface that achieves
    high performance on large clusters of commodity PCs."
  },
  "Dataset": {
    "answer": null,
    "evidence": null
  },
  "EvaluationMetric": {
    "answer": "elapsed time",
    "evidence": {
      "section": "Effect of Backup Tasks",
      "quote":
        "The entire computation takes 1283 seconds, an increase of
        44% in elapsed time."
    }
  }
}

```

Figure A3: Full extraction output for MapReduce [D2].

```

{
  "TechnicalMethod": {
    "answer": "Google",
    "evidence": {
      "section": "Abstract",
      "quote": "In this paper, we present Google, a
        prototype of a large-scale search engine which makes heavy
        use of the structure present in hypertext."
    }
  },
  "Task": {
    "answer": "information retrieval",
    "evidence": {
      "section": "Introduction",
      "quote": "The Web creates new challenges for
        information retrieval."
    }
  }
}

```

```

    },
    "Dataset": {
      "answer": "24 million pages",
      "evidence": {
        "section": "Anchor-test",
        "quote": "In our current crawl of 24 million pages.
we had over 259 million anchors which we indexed."
      }
    },
    "EvaluationMetric": {
      "answer": null,
      "evidence": null
    }
  }
}

```

Figure A4: Full extraction output for Google Search (PageRank) [D1].

```

{
  "TechnicalMethod": {
    "answer": "deep residual learning framework",
    "evidence": {
      "section": "Introduction",
      "quote": "In this paper, we address the degradation
problem by introducing a deep residual learning
framework."
    }
  },
  "Task": {
    "answer": "image classification",
    "evidence": {
      "section": "Introduction",
      "quote": "Deep convolutional neural networks [22,
21] have led to a series of breakthroughs for image
classification [21, 49, 39].\"
    }
  },
  "Dataset": {
    "answer": "ImageNet 2012 classification dataset",
    "evidence": {
      "section": "ImageNet Classification",
      "quote": "We evaluate our method on the ImageNet

```

```

2012 classification dataset [35] that consists of 1000
classes."
    }
  },
  "EvaluationMetric": {
    "answer": "top-1 and top-5 error rates",
    "evidence": {
      "section": "ImageNet Classification",
      "quote": "We evaluate both top-1 and top-5 error
rates."
    }
  }
}

```

Figure A5: Full extraction output for ResNet [D4].

Gold labels (six papers)

Paper	Gold TechnicalMethod	Gold Task	Gold Dataset	Gold Evaluati onMetri c
Transfo rmer [D6]	"Transformer"	"machin e translati on"	"WMT"	"BLEU"
BERT [D3]	"BERT"	"GLUE" or "SQuAD "	"BooksCor pus" or "Wikipedia "	"F1" or "accurac y"
AlexNet [D5]	"convolutional" (pa per predates the name "AlexNet")	"object recogniti on"	"ImageNet "	"top-1" or "top-5"
ResNet [D4]	"residual"	"image recogniti on"	"ImageNet "	"top-1"

Paper	Gold TechnicalMethod	Gold Task	Gold Dataset	Gold Evaluati onMetri c
MapRe duce [D2]	"MapReduce"	"distribu ted"	"TeraSort"	"seconds "
Google Search [D1]	"PageRank"	"web search"	"million pages"	"quality"

Table A1: Gold labels used for evaluation (six papers).

proto2 background data

Paper	TM	Tas k	Datase t	EM	Hypothesis set
Transformer [D6]	14	0	0	16 0	verbose_v1
BERT [D3]	62	23	15	13	short
AlexNet [D5]	51	6	11	4	short
ResNet [D4]	51	6	14	12	short
MapReduce [D2]	15 1	24	3	5	short
Google Search [D1]	69	21	8	29	short

Table A2: proto2 sentence-count output per role (six papers). See Table A3 for the per-paper pass/fail breakdown.

Paper	TechnicalM ethod	Tas k	Data set	Evaluation Metric	Total
	○	✗	○	✗	2/4

Paper	TechnicalMethod	Task	Dataset	Evaluation Metric	Total
Transformer					
BERT	○	○	○	○	4/4
AlexNet	○	○	○	○	4/4
ResNet	○	✗	○	○	3/4
MapReduce	○	✗	✗	○	2/4
Google Search	✗	○	○	○	3/4
Total					18/24 (75%)

Table A3: proto2 extended gold-label evaluation results (substring match, six papers). ML papers (Transformer, BERT, AlexNet, ResNet) scored 13/16 (81%); systems papers (MapReduce, Google Search) scored 5/8 (63%). ResNet scored ✗ on Task because "image recognition" does not appear in the 6 accepted Task sentences, likely because the paper frames the task as a competition result rather than an explicit label. MapReduce scored ✗ on Task and Dataset because "distributed" and "TeraSort" are absent from accepted sentences, consistent with the lack of standard ML benchmark structure. Google Search scored ✗ on TechnicalMethod because "PageRank" does not appear in any of the 69 accepted TechnicalMethod sentences, suggesting the algorithm name is mentioned in sentences classified as other roles.

The manual review is summarised in Chapter 5, Section 4 (Table 12). Variant B and the Task pilot outputs are in proto3/results_b/ and proto3/results_*/ in the repository.

Appendix B — PDF to TEI XML Conversion with GROBID

The conversion is a preprocessing step outside the notebook pipeline, which takes TEI XML as its input. I run GROBID 0.8.1 [11] locally in Docker, because a public GROBID server on HuggingFace failed in my test. The server starts with `docker run -d --rm -p 8070:8070 --name grobid lfoppiano/grobid:0.8.1`, and it is ready when `http://localhost:8070/api/isalive` responds. A short Python script posts each PDF to `http://localhost:8070/api/processFulltextDocument` (timeout 120 s) and saves the response as a TEI XML file. `docker stop grobid` stops the server. GROBID keeps the reference list outside the body and assigns section headings, which Stage 0 needs.